

Group Architecture & Security Design

Adding a secure Groups feature to ConnectX, and closing the underlying connection/permission gap that today lets any user message any other user.

SCOPE OF THIS DOCUMENT

This is an architecture and design specification only. No source code was modified, no database migration was created, and no schema, authentication, Web Push, or E2EE behavior was changed while producing it. Every claim about "what exists today" was verified by direct inspection of the ConnectX repository on 2026-08-17; every new component is explicitly labeled as a recommendation.

Document: ConnectX_Group_Architecture_Design.pdf

Repository: ConnectX — The Secure Messaging Service

Branch inspected: feature/pwa-notifications

Date: 2026-08-17

Prepared for: Vamsi

Table of Contents

1. Executive Summary
2. Current ConnectX Architecture
3. Current Security Gap
4. Proposed Relationship Architecture
5. Existing Chat Migration Strategy
6. Group Architecture
7. Group Invitation System
8. Group Roles & Permissions
9. Database / ER Diagram
10. REST API Design
11. WebSocket Architecture
12. Frontend Architecture
13. Group E2EE Architecture
14. Security Threat Model
15. Performance Considerations
16. Migration Strategy
17. Testing Strategy
18. Phased Implementation Roadmap
19. Risks and Alternatives
20. Final Recommended Architecture

Legend used throughout: **EXISTING** verified in the current codebase **RECOMMENDATION** new design proposed by this document
RISK a known gap or hazard that must be handled carefully **PHASE** a staged/future capability.

1. Executive Summary

ConnectX today lets any authenticated user search for any other user and immediately start an encrypted direct conversation with them — there is no connection request, no accept/reject step, and no blocking. This document designs a **Groups** feature, but ordering matters: building groups on top of today's "search = message" model would simply let a stranger add you to a group instead of DMing you directly, which is worse, not better. So this document first designs a **Connections & Blocking layer** that the whole app is missing, migrates existing conversations onto it losslessly, and only then builds Groups on top of it.

WHAT EXISTS TODAY (VERIFIED)

A Spring Boot 3 / MySQL backend with JWT auth, STOMP/WebSocket real-time delivery, and a single Conversation + ConversationMember model that already carries a DIRECT/GROUP type enum — but GROUP is dead code today; every code path that creates conversations hardcodes DIRECT. There is no group table, no invitation table, no connection table, and no block table anywhere in the schema. Direct messaging, media, reactions, stars, pins, edits, read/delivery receipts, typing indicators, presence, and Web Push all work and are reasonably mature. E2EE is pairwise ECDH (P-256) + AES-256-GCM between exactly two users' keys, but per the prior architecture audit it is not genuinely end-to-end (private key escrowed server-side, only TEXT messages are encrypted) — this is a pre-existing gap, not something this document proposes to fix, but it directly bounds what "Group E2EE" can honestly claim (Section 13).

WHAT THIS DOCUMENT RECOMMENDS

1. **Connections** (connection_requests + connections): PENDING/ACCEPTED/REJECTED/CANCELLED request workflow, collapsing to a fast, symmetric "are these two users connected" lookup table.
2. **Blocking** (user_blocks): directional block table enforced server-side at every messaging, invitation, and connection boundary.
3. **Lossless backfill**: every pair of users who already share a DIRECT conversation is migrated into an ACCEPTED connection automatically, with zero writes to messages, conversations, or any ciphertext column, and zero disruption to existing chats.
4. **Groups built on the existing Conversation model** (activating the dormant GROUP type) rather than a parallel model — reusing message loading, pagination, receipts, reactions, pinning, media, caching, and push infrastructure essentially unchanged.
5. **Group invitations require an existing accepted connection** between inviter and invitee (Option A, Section 7) — closing the exact same "stranger adds you" abuse vector for groups that Connections closes for DMs.
6. **A mandatory WebSocket fix** (Section 11): the existing /topic/conversation/{id} broadcast has no subscribe-time membership check today. For a 2-person DM this leaks one conversation; for a group it leaks every member's traffic to any authenticated user who guesses an ID. This must be closed *before* group messaging ships, not deferred to a later stage.
7. **Phased Group E2EE** (Section 13): Phase 1 ships group membership only (no group messages yet); Phase 2 extends the existing pairwise-ECDH primitive with per-recipient fan-out encryption (still genuinely E2E, no forward secrecy); Phase 3 is the future upgrade to a proper sender-key/ratchet scheme, explicitly gated on first fixing the existing 1:1 identity-key escrow problem, since group crypto cannot be stronger than the identity keys it is built on.

The remainder of this document traces the actual code behind every claim above, then works through the relationship model, migration strategy, group domain model, invitation flow, roles, database design, REST/WebSocket APIs, frontend integration, E2EE phasing, threat model, and a ten-stage implementation roadmap designed so that at every stage the existing, working messaging behavior keeps working.

2. Current ConnectX Architecture

This section documents what exists today, traced directly from the repository. Nothing in this section is a recommendation.

2.1 Backend module layout

Spring Boot 3, package-by-feature under `com.connectx`: `auth`, `user`, `device`, `conversation`, `message`, `media`, `push`, `websocket`, `config`, `common`. Each feature package follows the same `controller / dto / entity / repository / service` sub-structure. There is no dedicated "connection" or "group" package today.

2.2 Core entities (verified field-by-field)

ENTITY	KEY FIELDS	NOTES
User	<code>id</code> , <code>username</code> , <code>email</code> , <code>passwordHash</code> , <code>displayName</code> , <code>profileImageUrl</code> , <code>status</code> , <code>lastSeenAt</code> , <code>masterPublicKey</code> , <code>masterPrivateKey</code> (TEXT, escrowed), <code>createdAt</code> , <code>updatedAt</code>	One row per account. <code>status</code> is "ONLINE"/"OFFLINE" driven by <code>PresenceService</code> .
Device	<code>id</code> , <code>user</code> , <code>deviceName</code> , <code>publicKey</code> , <code>keyAlgorithm</code> , <code>createdAt</code> , <code>lastSeenAt</code> , <code>active</code>	Per-device public key registered separately from the user's master key; per prior audit, device "revocation" is cosmetic (nothing keys off <code>active=false</code> during encryption).
Conversation	<code>id</code> , <code>type</code> (DIRECT GROUP), <code>createdAt</code> , <code>updatedAt</code> , <code>members</code>	The GROUP enum value already exists (<code>ConversationType.java:5</code>) but is never referenced anywhere else in the backend — confirmed via full-repo grep. Dead/placeholder code.
ConversationMember	<code>id</code> , <code>conversation</code> , <code>user</code> , <code>joinedAt</code> , <code>deletedAt</code> , <code>clearedAt</code> , <code>lastReadMessageId</code> , <code>pinned</code> , <code>pinnedAt</code> , <code>mutedUntil</code> , <code>archived</code> , <code>archivedAt</code> , <code>manuallyMarkedUnread</code>	Per-user, per-conversation state. No <code>role</code> column exists today — every member of every conversation is structurally equal.
Message	<code>id</code> , <code>conversation</code> , <code>senderUser</code> , <code>senderDevice</code> , <code>recipientDevice</code> , <code>messageType</code> , <code>mediaId</code> , <code>caption</code> , <code>latitude/longitude/locationLabel</code> , <code>encryptionAlgorithm</code> , <code>ciphertext</code> , <code>nonce</code> , <code>sentAt</code> , <code>deliveredAt</code> , <code>readAt</code> , <code>deletedForEveryone</code> , <code>deletedAt</code> , <code>replyToMessage</code> , <code>editedAt</code> , <code>forwarded</code> , <code>pinnedAt</code> , <code>pinnedBy</code> , <code>reactions</code>	<code>recipientDevice</code> is singular — the schema already assumes one sender and, at most, one specific recipient device per row, never a fan-out to many recipients. This matters directly for Section 13.
MessageUserState	<code>id</code> , <code>message</code> , <code>user</code> , <code>deletedAt</code>	Per-user "deleted for me" marker, independent of the shared message row.
MessageReaction / MessageStar / MessageMedia	—	All keyed by <code>message_id/conversation_id</code> , already generic over member count (see 2.5).

2.3 Direct-conversation creation and messaging (as implemented)

VERIFIED FLOW

`ConversationService.createOrGetDirectConversation()` (`ConversationService.java:67-119`) takes a `targetUserId`, checks only that the target user exists, looks up (or creates) a `Conversation(ConversationType.DIRECT)` via `ConversationRepository.findDirectConversationBetweenUsers()`, and inserts two `ConversationMember` rows. There is no relationship, request, or permission check of any kind between the two users — this is the exact gap addressed in Section 3.

`MessageService.sendMessage()` (`MessageService.java:98-312`) only checks that the sender is a member of the target conversation (`conversationMemberRepository.findByConversationIdAndUserId`) — it never checks any relationship between sender and other members. This check is already conversation-member-based rather than "exactly 2 people" based, which is favorable for Section 6.

2.4 Encryption as actually implemented (backend side)

`Message.encryptionAlgorithm` defaults to "ECDH-P256+AES-256-GCM" for TEXT messages; IMAGE/LOCATION/DOCUMENT messages are stored with `encryptionAlgorithm="NONE"` and empty `ciphertext/nonce` (`MessageService.java:154-193`) — captions on those message types are stored as plaintext in `messages.caption`. The backend never decrypts anything itself; it stores and relays opaque `ciphertext/nonce` strings for TEXT messages only. `User.masterPrivateKey` is a real column that the client actively reads and writes via `UserService.getIdentityKey()/saveIdentityKey()` (`UserService.java:196-215`) — i.e., the "E2E" private key is genuinely persisted server-side today. This was previously identified in the prior architecture audit and is out of scope to fix here, but it is the ceiling on what "Group E2EE" can honestly promise (Section 13.4).

2.5 WebSocket / STOMP architecture

Endpoint/ws (SockJS + raw), broker prefixes /topic and /queue, app prefix /app, user prefix /user Auth today CONNECT-time only — `WebSocketAuthChannelInterceptor.java:29-42` validates the JWT on the STOMP CONNECT frame and attaches a `UsernamePasswordAuthenticationToken` as the session principal. **No interceptor logic exists for the SUBSCRIBE command at all** — confirmed by full-repo grep for `StompCommand.SUBSCRIBE`, zero matches outside the CONNECT branch. Broadcast pattern Every mutating message operation (`sendMessage`, `deleteMessage`, `editMessage`, `pin/unpin`, `reactions`) does **both**: (a) `messagingTemplate.convertAndSend("/topic/conversation/"+id, event)` — a public STOMP topic with no per-subscriber check, and (b) a loop over `conversationMemberRepository.findByConversationId...WithUsers()` calling `convertAndSendToUser(username, "/queue/messages", event)` for each member individually. The per-user queue send is what Spring's user-destination mechanism actually scopes safely (resolved server-side to a session-specific queue); the topic send has no equivalent protection. Client-send destinations/app/message.send, /app/message.delivered, /app/message.read, /app/typing Typing / presence `WebSocketMessageController.handleTyping()` (`WebSocketMessageController.java:168-210`) already loops over *all* members of the conversation, not a hardcoded pair — already N-member-safe with zero changes needed. `PresenceService` broadcasts ONLINE/OFFLINE to every user who shares *any* conversation with the affected user, also already member-count-agnostic.

RISK CARRIED FORWARD FROM THE PRIOR AUDIT — STILL PRESENT

The missing SUBSCRIBE-time authorization on `/topic/conversation/{id}` means any authenticated user who learns or guesses a conversation ID can subscribe to it and receive every future `MESSAGE_RECEIVED/MESSAGE_EDITED/reaction/pin` event broadcast to that topic, bypassing the membership check entirely (the check exists on REST endpoints and on the private queue fan-out, but not on the public topic subscription itself). This was independently found by two tracks of the 2026-08-16 audit and is still unresolved as of this inspection. See Section 11 for why this must be fixed before Groups, not after.

2.6 Frontend module layout

See Section 12 for the full, verified frontend trace (state architecture, WebSocket client lifecycle, caching, crypto, PWA/notifications). Summarized here for continuity: React + TypeScript SPA (`connectx-frontend/src`), `App.tsx` as the top-level state owner, a singleton STOMP client

(websocket/WebSocketClient.ts), IndexedDB-backed crypto key storage and message cache, and a hand-rolled service worker (public/sw.js, no build-time PWA plugin) handling Web Push and share-target.

3. Current Security Gap

THE GAP, PRECISELY

GET /api/v1/users/search?username= (**UserController.java:26-30** → **UserService.searchUsersByUsername()**) returns any matching user to any authenticated caller with no relationship check whatsoever. POST /api/v1/conversations/direct (**ConversationController.java:32-38** → **ConversationService.createOrGetDirectConversation()**) then creates a live, message-capable conversation from nothing but that target user's ID — again with no relationship check. The two endpoints compose directly: search, pick a result, POST the ID, you are now messaging a stranger. There is no PENDING state, no accept step, and nothing to reject or block.

This is the root problem the user asked to correct before Groups are introduced, and it generalizes directly: whatever mechanism lets a stranger start a 1:1 chat with you today would, without a fix, let a stranger add you to a group tomorrow — group membership is even more exposed than a DM, since it also broadcasts your presence and messages to every other member, not just one person.

GOVERNING PRINCIPLE FOR THE REST OF THIS DOCUMENT

User discovery ≠ connection ≠ messaging permission ≠ group membership. Search (discovery) must never, by itself, grant the ability to message someone (permission) or add them to a group (membership). Each of those is a distinct, explicitly-granted relationship state, enforced server-side.

4. Proposed Relationship Architecture

RECOMMENDATION

4.1 Connections: one workflow table + one fast lookup table

Two options were weighed: a single table carrying both the request workflow and the final relationship state, versus splitting "the act of requesting" from "the fact of being connected." ConnectX's own patterns favor the split: every hot-path authorization check in the existing code (`existsByConversationIdAndUserIdAndDeletedAtIsNull`, etc.) is a single indexed boolean lookup, not a status-filtered scan. A connection check will run on *every* DM start, every group invitation, and potentially every message send once blocking is enforced — it needs to be that same shape of query.

TABLE	PURPOSE & SHAPE
connection_requests	The workflow/audit trail. One row per request attempt: <code>requester_id</code> , <code>recipient_id</code> , <code>status</code> (<code>PENDING ACCEPTED REJECTED CANCELLED</code>), <code>created_at</code> , <code>responded_at</code> . A new request after a <code>REJECTED/CANCELLED</code> terminal state inserts a new row rather than reusing the old one, preserving history (mirrors how <code>otpToken</code> already keeps a full row per attempt rather than overwriting). A partial unique index on (<code>requester_id</code> , <code>recipient_id</code>) where <code>status='PENDING'</code> prevents duplicate simultaneous requests in either direction.
connections	The fast, symmetric fact table: one row per <i>pair</i> , using canonical ordering (<code>user_id_low = LEAST(a,b)</code> , <code>user_id_high = GREATEST(a,b)</code>) so "are these two connected" is a single unique-indexed lookup regardless of which user initiated it. Created the instant a request transitions to <code>ACCEPTED</code> , or by the one-time backfill (Section 5). Carries <code>created_at</code> and a source flag (<code>REQUEST</code> vs <code>MIGRATED</code>) purely for auditability.

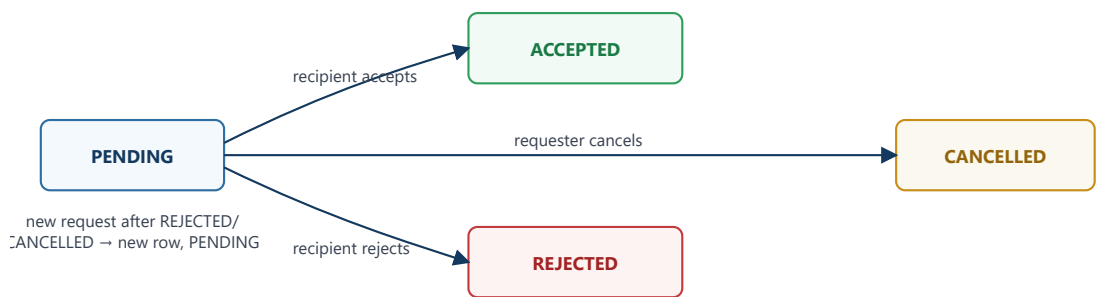


Fig. 4.1 — `connection_requests` status lifecycle. `ACCEPTED` additionally writes one row into the symmetric `connections` table.

4.2 Blocking

TABLE	SHAPE
user_blocks	<code>id</code> , <code>blocker_id</code> , <code>blocked_id</code> , <code>created_at</code> , unique on (<code>blocker_id</code> , <code>blocked_id</code>). Directional and independent of connections — blocking someone you're connected to does not delete the connection row (so an unblock doesn't need to re-request); it simply gates enforcement (below) on top of it.

4.3 Blocking effects — enforced server-side, endpoint by endpoint

SURFACE	EFFECT OF A BLOCK (EITHER DIRECTION)
Direct messaging	<code>MessageService.sendMessage()</code> and the <code>WS /app/message.send</code> path add a block check (<code>existsByBlockerAndBlocked</code> , checked both directions) alongside the existing membership check, before the message is persisted. Rejected with 403 BLOCKED.
Connection requests	Sending a request to a user who has blocked you (or whom you've blocked) is rejected at the API layer. Blocking a user with a PENDING request in either direction auto-transitions that request to CANCELLED.
Group invitations	An inviter cannot invite a user who has blocked them, or whom they've blocked — checked in the same service call as the connection-required check (Section 7).
Existing conversations	The DIRECT conversation and its message history are not deleted or hidden by a block — only new sends are rejected, symmetric to how "delete for me" already works today without destroying the other party's copy.
Typing indicators / read receipts	Both already fan out per-recipient (<code>handleTyping</code> , <code>markRead/markDelivered</code>); the same block check used for sends is applied so a blocked pair stops exchanging live signals, not just messages.
User search	Recommendation: search results are unaffected by blocks in either direction. Discovery is not messaging permission (Section 3's governing principle) — and filtering search by block status would itself leak "you can no longer find this person," which is a detectable block-status side-channel. All enforcement happens at the messaging/invitation boundary instead, where a clear, intentional error is appropriate.
Notifications	Since a block prevents the message from ever being persisted, no push/WS event is ever generated toward the blocked pair — enforcement at the write boundary makes downstream notification filtering unnecessary.

5. Existing Chat Migration Strategy

RECOMMENDATION

NON-NEGOTIABLE CONSTRAINTS (FROM DIRECT INSPECTION)

ConnectX uses Hibernate `ddl-auto: update` (`application.yml:27`) with **no Flyway or Liquibase anywhere in the repository** (confirmed by search). There is no existing migration history to preserve or break — but it also means there is no existing mechanism for a one-time *data* backfill; `ddl-auto` only handles schema shape, never data. Any backfill script is new infrastructure, which is exactly why it must be designed to be safe to run more than once and safe to run late.

5.1 The backfill rule



Fig. 5.1 — every existing *DIRECT* conversation's member pair becomes an *ACCEPTED* connection, unconditionally.

For every distinct pair of users who share a row in `conversation_members` under a conversation with `type = DIRECT` (read directly from `conversation_members` joined to `conversations`, regardless of either member's per-user `deleted_at/archived/cleared_at` flags — those are visibility flags, not relationship flags), insert one `connections` row with `source = MIGRATED`, and one corresponding `connection_requests` row with `status = ACCEPTED` for audit continuity (requester defaults to whichever member has the earlier `joined_at`, an arbitrary but stable choice since both members already have full messaging history together).

5.2 Why this is safe

GUARANTEE	HOW IT'S MET
Messages untouched	The backfill reads <code>conversation_members</code> and writes only to the two new tables. It never touches <code>messages</code> , <code>message_reactions</code> , <code>message_stars</code> , or <code>message_media</code> .
Ciphertext untouched	No column on <code>messages.ciphertext/nonce</code> is read or written by the backfill.
Conversation IDs stable	No <code>conversations</code> row is inserted, deleted, or has its <code>id</code> touched.
Receipts/WS keep working	Read/delivery receipts and WS delivery key off <code>conversation_id/conversation_members</code> , neither of which the backfill modifies.
No forced re-acceptance	Because the backfill runs <i>before</i> the "connection required to message" enforcement is turned on (Section 18, Stage 2), by the time enforcement is live every existing pair is already <i>ACCEPTED</i> — no existing user is ever shown a pending request for someone they already talk to.
Idempotent	The backfill is a plain <code>INSERT ... SELECT ... WHERE NOT EXISTS</code> (matching the existing native-query style already used in <code>MessageUserStateRepository.bulkInsertDeletedStateForConversation</code>) guarded additionally by a one-row marker table (<code>schema_backfills(name, completed_at)</code>) so it is safe to invoke on every boot without rescanning millions of rows once complete.

5.3 Recommended mechanism

RECOMMENDATION: INTRODUCE FLYWAY AT STAGE 0

Today, schema changes happen implicitly via `ddl-auto: update`, which cannot express a data backfill and leaves no record of what ran when — a latent operational risk independent of Groups. This document recommends introducing Flyway starting with the very first Groups-related schema change (new tables are additive and low-risk either way), so the connections/blocking/groups tables *and* the backfill are captured as ordered, checksummed, re-runnable migration scripts rather than an ad hoc boot-time runner. This is a tooling recommendation with real but bounded effort; the fallback (a guarded idempotent `ApplicationRunner` bean, no new dependency) is noted as an alternative in Section 19 if introducing Flyway is judged out of scope for this initiative.

6. Group Architecture

RECOMMENDATION

6.1 Conversation model decision

RECOMMENDATION: EXTEND THE EXISTING CONVERSATION/CONVERSATIONMEMBER MODEL — DO NOT BUILD A PARALLEL MODEL

- `ConversationType.GROUP` already exists in the enum (`ConversationType.java:5`), unused — a strong signal this was the intended direction.
- Message loading/pagination (`MessageRepository.findVisibleMessagesPaged`), read/delivery receipts, reactions, stars, pins, edits, media, and the conversation-list N+1 fix (`findLatestMessagesForConversations`) are already keyed purely by `conversation_id` and member count, with no DIRECT-specific assumption baked in.
- Typing indicators and presence fan-out are already member-list-based, not pair-based (Section 2.5) — they work for groups today, with zero backend changes.
- The only genuinely DIRECT-specific code is narrow and easy to leave alone: `createOrGetDirectConversation()`, `findDirectConversationBetweenUsers()`, and the "last member out hard-deletes the conversation" branch in `deleteConversationForUser()` (`ConversationService.java:326-389`), which needs a group-aware variant (Section 6.4).

6.2 New tables

TABLE	PURPOSE
groups	Group-specific metadata, one row per group conversation. Shares its primary key with <code>conversations.id</code> (i.e. <code>groups.conversation_id</code> is both PK and FK) rather than having its own identity — keeps "conversationId" the single universal handle the rest of the app (frontend routing, WS topics, caches) already uses everywhere.
group_invitations	The invite/accept workflow (Section 7).

groups — fields

`conversation_id` PK, FK → `conversations.id` name VARCHAR(100), required description VARCHAR(500), optional avatar_url nullable, same storage pattern as `User.profileImageUrl` `created_by_user_id` FK → `users.id`, the founding OWNER `who_can_invite` enum, default OWNER_ADMIN_ONLY (Section 8.4) `created_at` / `updated_at` standard

6.3 Extending ConversationMember

Two nullable, additive columns — zero risk to existing DIRECT rows, which simply carry NULL in both:

role nullable enum OWNER|ADMIN|MEMBER; NULL for every DIRECT membership row (a DIRECT conversation has no roles)
`invited_by_user_id` nullable FK → `users.id`; records who actually added this member, for audit and for the "who can remove whom" rule in Section 8

6.4 Group-specific behavior that must diverge from DIRECT

EXISTING DIRECT BEHAVIOR	REQUIRED GROUP-AWARE CHANGE
<code>deleteConversationForUser</code>: when the last active member soft-deletes, the conversation and all its rows are hard-deleted (<code>ConversationService.java:350-381</code>).	For groups, "delete for me" (a member leaving/hiding the group from their own list) must not hard-delete the group out from under remaining members. Recommendation: branch on <code>conversation.type</code>; group deletion is only ever explicit (<code>DELETE /groups/{id}</code>, OWNER-only, Section 10), never an emergent side-effect of the last visible member leaving. A member's own "leave" always uses the existing soft-delete-for-self path.
<code>createOrGetDirectConversation</code> is idempotent by design (same two users always resolve to the same conversation).	Group creation is never idempotent/resolved by member set — every <code>POST /groups</code> call creates a new group, even with identical members. No change needed, just a note that the two creation paths must stay separate (they already will be, since group creation is new code, not a variant of <code>createOrGetDirectConversation</code>).

7. Group Invitation System

RECOMMENDATION

7.1 Option comparison

OPTION	DESCRIPTION	VERDICT
A	Only existing accepted connections can be invited to a group.	Recommended. See below.
B	Any searchable user can be invited, regardless of connection status.	Rejected — this is the exact same "search = permission" flaw from Section 3, just relocated to groups. A group invite is arguably worse than a stray DM: acceptance exposes the invitee to every current and future member, not one person.
C	Configurable per-group privacy (open vs. connections-only vs. admin-approval).	Reasonable future extension, not recommended as Phase-1 scope — it multiplies the authorization logic and the abuse surface for no validated user need yet. Revisit once Groups ships and real usage patterns are known (Section 19).

RECOMMENDATION: OPTION A, WITH THE CONNECTION REQUIRED BETWEEN THE SPECIFIC INVITER AND INVITEE

The rule is not "the invitee is connected to *someone* in the group" — it is "the inviting OWNER/ADMIN has an ACCEPTED connection with the invitee," checked with the exact same connections lookup used for starting a DM. This keeps the rule auditable (one admin's action, one admin's connection graph) and prevents a scenario where a compromised or careless admin adds a stranger by piggy-backing on a different member's connection.

7.2 Flow

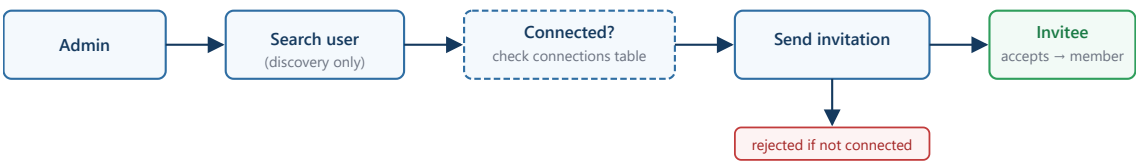


Fig. 7.1 — group invitation flow. Search never bypasses the connection check.

7.3 group_invitations table

idPK group_idFK → conversations.id (the group) invitee_user_idFK → users.id invited_by_user_idFK → users.id (must hold OWNER/ADMIN role at send time) statusPENDING|ACCEPTED|REJECTED|CANCELLED created_at / responded_atstandard

Unique partial index on (group_id, invitee_user_id) where status='PENDING' prevents duplicate outstanding invitations to the same person. Acceptance is the only path that inserts a conversation_members row with role=MEMBER for a group — there is no other way to become a group member, mirroring "no DM without an accepted connection."

8. Group Roles & Permissions

RECOMMENDATION

8.1 Roles

OWNER (exactly one at all times), **ADMIN** (zero or more), **MEMBER** (zero or more). Stored in the new `conversation_members.role` column (Section 6.3).

8.2 Permission matrix

ACTION	OWNER	ADMIN	MEMBER	NOTES
Create group	Any authenticated user			Creator becomes OWNER automatically.
Rename group / change avatar / description	✓	✓	—	
Invite members	✓	✓	—	Subject to <code>who_can_invite</code> group setting (default <code>OWNER_ADMIN_ONLY</code> , see 8.4).
Cancel a pending invitation	✓	✓	—	Any OWNER/ADMIN can cancel any pending invitation, not only their own — a shared moderation responsibility.
Remove a MEMBER	✓	✓	—	
Remove an ADMIN	✓	—	—	ADMINS cannot remove each other — prevents an admin power struggle; only the OWNER can.
Remove the OWNER	Never possible			Ownership must be transferred first.
Promote MEMBER → ADMIN	✓	—	—	OWNER-only, deliberately — keeps the power structure a simple two-level delegation, not a chain.
Demote ADMIN → MEMBER	✓	—	—	
Transfer ownership	✓	—	—	Target becomes OWNER; previous OWNER becomes ADMIN (not removed).

ACTION	OWNER	ADMIN	MEMBER	NOTES
Leave group	Only after transferring ownership (or if last member remaining, which deletes the group)	✓	✓	
Delete group	✓	—	—	Explicit, irreversible; distinct from any member leaving (Section 6.4).
Accept / reject an invitation addressed to them	The invitee only			No one else can accept on another user's behalf.
Send group messages, react, reply, star	✓	✓	✓	Identical to today's per-member permissions in a DIRECT conversation — no role gating on ordinary messaging actions.

8.3 Enforcement pattern

Matches the existing codebase's approach exactly: **no @PreAuthorize annotations exist anywhere in ConnectX today** (confirmed by repo-wide search); every authorization check is an explicit, imperative lookup inside the service method (e.g. `conversationMemberRepository.existsByConversationIdAndUserIdAndDeletedAtIsNull()`). Group role checks should follow the same style — a small `GroupAuthorizationService` (or private helper methods on a new `GroupService`, mirroring how `MediaService.ensureConversationMember()` is a private helper reused across several public methods) doing `requireRole(groupId, userId, Set.of(OWNER, ADMIN))`-style checks, throwing the existing `ApiException` with a 403 status and a specific error code.

8.4 Group settings (minimal, Phase-1 scope)

`who_can_invite`: `OWNER_ADMIN_ONLY` (default) or `ALL_MEMBERS`. This is the only configurable setting in Phase 1; it is intentionally narrow. Broader "group privacy" (Option C from Section 7.1) is deferred.

9. Database / ER Diagram

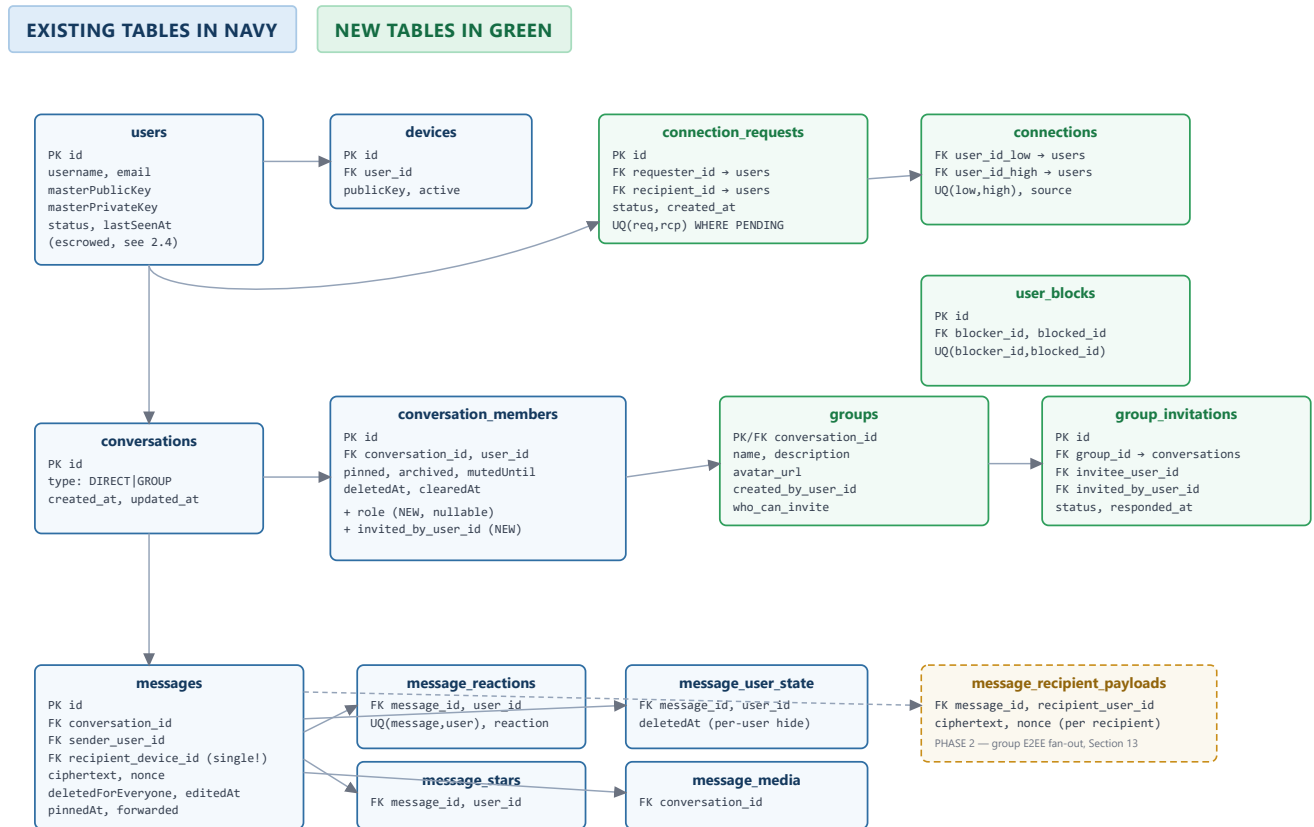


Fig. 9.1 — full entity-relationship diagram. Navy = existing (unchanged unless noted); green = new tables/columns recommended by this document; amber dashed = Phase 2 (Section 13), not built in Phase 1.

9.1 Constraints, cascades, and indexes for the new tables

TABLE	INDEXES / CONSTRAINTS	DELETE BEHAVIOR
connection_requests	Index on (recipient_id, status) for "my incoming requests"; partial unique on (requester_id, recipient_id) WHERE PENDING.	Rows are never hard-deleted (audit trail); terminal states are final.
connections	Unique on (user_id_low, user_id_high); supporting index on each column individually for "all my connections" queries.	Deleted only if a future "remove connection" feature is added (out of scope for Phase 1); FK ON DELETE CASCADE from users so account deletion cleans up automatically (matches no explicit account-deletion feature existing today, but keeps the FK sound).
user_blocks	Unique on (blocker_id, blocked_id); index on blocked_id alone for the enforcement checks that don't know direction.	ON DELETE CASCADE from users.

TABLE	INDEXES / CONSTRAINTS	DELETE BEHAVIOR
groups	PK is the shared conversation_id — no separate index needed for group lookup by conversation.	ON DELETE CASCADE from conversations, mirroring how conversation_members already cascades from Conversation via CascadeType.ALL, orphanRemoval=true (Conversation.java:26).
group_invitations	Partial unique on (group_id, invitee_user_id) WHERE PENDING; index on invitee_user_id for "my incoming group invitations."	ON DELETE CASCADE from both conversations (group deleted) and users (invitee/inviter account deleted).

9.2 Transaction boundaries

Following the existing codebase's pattern exactly (service-method-level `@Transactional`, with the proven `REQUIRES_NEW`-plus-self-proxy pattern already used for `insertReactionInNewTransaction` and `insertStarInNewTransaction` for races on a unique constraint): accepting a connection request is one transaction that both updates `connection_requests.status` and inserts the `connections` row; accepting a group invitation is one transaction that updates `group_invitations.status` and inserts the `conversation_members` row. Section 15 covers the specific race (simultaneous mutual requests) this pattern must guard against.

10. REST API Design

RECOMMENDATION — NOT IMPLEMENTED

All endpoints follow the existing conventions verified in **ConversationController/MessageController**: JWT bearer auth via the existing `JwtAuthenticationFilter` (nothing new required for authentication itself), responses wrapped in the existing `ApiResponse<T>` envelope, errors via the existing `ApiException(status, code, message) → ErrorResponse` path handled by `GlobalExceptionHandler`. No new authentication mechanism is introduced.

10.1 Connections

METHOD & URL	AUTHZ	BODY / PARAMS	KEY ERRORS
POST /api/v1/connections/requests	Authenticated; reject if blocked either direction	{targetUserId}	400 SELF_REQUEST, 403 BLOCKED, 409 ALREADY_CONNECTED, 409 REQUEST_ALREADY_PENDING
GET /api/v1/connections/requests? direction=incoming outgoing	Authenticated, self- scoped	query param	—
POST /api/v1/connections/requests/{id}/accept	Caller must be the recipient_id of that request	—	403 FORBIDDEN, 404 REQUEST_NOT_FOUND, 409 REQUEST_NOT_PENDING
POST /api/v1/connections/requests/{id}/reject	Caller must be the recipient	—	same as above
POST /api/v1/connections/requests/{id}/cancel	Caller must be the requester	—	same as above
GET /api/v1/connections	Authenticated, self- scoped	—	—

10.2 Blocking

METHOD & URL	AUTHZ	BODY	KEY ERRORS
POST /api/v1/blocks	Authenticated	{userId}	400 SELF_BLOCK
DELETE /api/v1/blocks/{userId}	Authenticated, self-scoped	—	—
GET /api/v1/blocks	Authenticated, self-scoped	—	—

10.3 Groups

METHOD & URL	AUTHZ	BODY	KEY ERRORS
POST /api/v1/groups	Authenticated (any user may create)	{name, description?, avatarUrl?}	400 VALIDATION_ERROR
GET /api/v1/groups/{groupId}	Caller must be a current member	—	403 NOT_GROUP_MEMBER, 404

METHOD & URL	AUTHZ	BODY	KEY ERRORS
PATCH /api/v1/groups/{groupId}	OWNER/ADMIN	{name?, description?, avatarUrl?, whoCanInvite?}	403 FORBIDDEN
DELETE /api/v1/groups/{groupId}	OWNER only	—	403 FORBIDDEN
POST /api/v1/groups/{groupId}/leave	Any member; OWNER blocked unless sole member	—	409 OWNER_MUST_TRANSFER_FIRST
GET /api/v1/groups/{groupId}/members	Current member	—	—
DELETE /api/v1/groups/{groupId}/members/{userId}	OWNER/ADMIN, role-hierarchy rule (8.2)	—	403 CANNOT_REMOVE_OWNER, 403 ADMIN_CANNOT_REMOVE_ADMIN
POST /api/v1/groups/{groupId}/members/{userId}/promote	OWNER only	—	403 FORBIDDEN
POST /api/v1/groups/{groupId}/members/{userId}/demote	OWNER only	—	403 FORBIDDEN
POST /api/v1/groups/{groupId}/transfer-ownership	OWNER only	{newOwnerUserId}	400 TARGET_NOT_MEMBER

10.4 Group invitations

METHOD & URL	AUTHZ	BODY	KEY ERRORS
POST /api/v1/groups/{groupId}/invitations	OWNER/ADMIN (or any member if who_can_invite=ALL_MEMBERS); inviter must have an ACCEPTED connection with invitee	{inviteeUserId}	403 NOT_CONNECTED, 403 BLOCKED, 409 ALREADY_MEMBER, 409 INVITATION_ALREADY_PENDING
GET /api/v1/groups/invitations? direction=incoming outgoing	Authenticated, self-scoped	—	—
POST /api/v1/groups/invitations/{id}/accept	Caller must be the invitee	—	409 INVITATION_NOT_PENDING
POST /api/v1/groups/invitations/{id}/reject	Caller must be the invitee	—	same as above
DELETE /api/v1/groups/invitations/{id}	Original inviter, or any OWNER/ADMIN of that group	—	—

10.5 Group messages

NO NEW ENDPOINTS NEEDED

Because Section 6 extends the existing Conversation model rather than creating a parallel one, group messaging reuses GET /api/v1/conversations/{id}/messages, POST /api/v1/messages, and every reaction/star/pin/edit/delete endpoint verbatim — the existing membership check (conversationMemberRepository.existsByConversationIdAndUserIdAndDeletedAtIsNull) already generalizes to N members with no code change. This is the single strongest argument for the Section 6.1 recommendation.

11. WebSocket Architecture

HARD PREREQUISITE — MUST BE FIXED BEFORE ANY GROUP MESSAGING SHIPS

As documented in Section 2.5, `/topic/conversation/{id}` has no subscribe-time membership check today. For a DIRECT conversation this exposes one other person's traffic to a guessing attacker; for a GROUP conversation the same hole exposes every member's traffic, and — because nothing revokes an existing subscription — a member who is *removed* from a group could keep receiving live messages for as long as their WebSocket session stays open. This is not a "Stage 7" concern; it is a precondition.

11.1 Two remediation options

OPTION	DESCRIPTION	ASSESSMENT
1. Fix the topic	Extend the existing <code>WebSocketAuthChannelInterceptor</code> (or add a second interceptor) to also handle <code>StompCommand.SUBSCRIBE</code> : parse the destination, extract the conversation ID, and reject the subscribe frame unless <code>conversationMemberRepository.existsByConversationIdAndUserIdAndDeletedAtIsNull</code> is true.	Necessary regardless, since the topic broadcast is already load-bearing for DIRECT conversations today and cannot simply be deleted without frontend verification (Section 12 traces whether the frontend actually subscribes to it).
2. Prefer the private queue	The parallel <code>convertAndSendToUser(username, "/queue/messages", event)</code> fan-out (already present in every mutating method) is inherently safe: Spring's user-destination mechanism resolves <code>/user/queue/messages</code> to a session-specific queue server-side, so one user structurally cannot subscribe to another's.	Complementary, not a substitute for Option 1: both mechanisms already fire on every event today, so the public topic's exposure exists independent of whether the private queue "already does the job." Option 1 must still be applied; Option 2 is a defense-in-depth note, not a way to defer Option 1.

Section 12 reports whether the frontend's WebSocket client actually depends on the `/topic/conversation/{id}` subscription, which determines whether Option 1 is strictly required for groups or whether the topic can be deprecated outright — see the frontend trace for the concrete answer.

11.2 New group event types

All follow the existing `WsEvent{type, requestId, payload}` envelope (`WsEvent.java`) and the existing dual-delivery pattern (private per-member queue, and the topic once 11.1 is fixed), fanned out via the same `afterCommitExecutor.runAfterCommit(...)` pattern already used for every existing broadcast (guaranteeing the event only fires after the DB transaction actually commits).

EVENT TYPE	FIRED WHEN	DELIVERED TO
GROUP_MEMBER_ADDED	Invitation accepted	All existing members (including the new one)
GROUP_MEMBER_REMOVED	Admin/owner removes a member	All remaining members <i>and</i> the removed user's private queue (so their own client can locally unsubscribe/exit the conversation view — see 11.1's residual-exposure note)
GROUP_MEMBER_LEFT	Member calls leave	Remaining members
GROUP_RENAMED / GROUP_AVATAR_CHANGED / GROUP_SETTINGS_CHANGED	OWNER/ADMIN edits group metadata	All current members
GROUP_ROLE_CHANGED	Promote/demote/ownership transfer	All current members
GROUP_INVITATION_RECEIVED	Invitation sent	Invitee's private queue only (not broadcast to the group — the invitee isn't a member yet and has no topic access)
GROUP_INVITATION_ACCEPTED / REJECTED / CANCELLED	Invitee/admin responds	The inviter's private queue; ACCEPTED also triggers GROUP_MEMBER_ADDED to the group
GROUP_DELETED	OWNER deletes the group	All members' private queues (mirrors the existing CONVERSATION_DELETED per-user pattern in <code>ConversationService.java:384-388</code>)

11.3 Reused, unmodified today

`/app/message.send`, `/app/message.delivered`, `/app/message.read`, `/app/typing` (already member-list-based, Section 2.5), and presence broadcast — all require zero backend changes for group support beyond the SUBSCRIBE fix in 11.1, since they already operate over "all members of conversation X," never "the other one person."

12. Frontend Architecture

12.1 What exists today (verified trace)

AREA	VERIFIED FACT
State ownership	App.tsx is the single owner of conversations, activeConversation, and messages (a flat array always scoped to whichever conversation is active, not a per-conversation map). Everything else (ChatScreen, MessageFeed, MessageInput) is prop-driven with no independent data fetching.
The 1:1 assumption's single root cause	getOtherParticipant() (App.tsx:62-66 : first member where user.id !== currentUserId) is independently reimplemented four times : App.tsx itself (15+ call sites), ChatListSidebar.tsx:53-59 (as getRecipient), ForwardMessageModal.tsx:15-19 , and a dead, unimported Sidebar.tsx:27 . Every one of these needs a group-aware replacement (12.2).
Sidebar filtering — concrete bug-to-be	ChatListSidebar.tsx:66-67 currently drops any conversation from the visible list entirely if getRecipient returns null (i.e. not exactly one other member). Left unfixed, a group conversation would silently vanish from the sidebar the day groups ship.
WebSocket client — already generic where it matters	WebSocketClient.ts subscribes to at most one /topic/conversation/{id} at a time, swapped whenever activeConversation changes (App.tsx:265 → wsClient.setActiveConversation(id)) — this mechanism is already purely numeric-id-keyed with no 1:1 assumption; it needs zero changes for groups. Separately, /user/queue/messages is subscribed once per connection and already receives <i>every</i> incoming message regardless of which topic is active — the per-conversation topic exists only for secondary live signals (typing, reactions, edits, pins, read receipts) while that conversation happens to be open on screen.
Data model readiness	Conversation.type in types/index.ts:72 already types 'GROUP' as a valid value (mirroring the backend's dead enum, Section 2.2) — confirmed the only reference to it anywhere in the frontend. ConversationMember[] is already array-shaped with per-member pinned/archived/mutedUntil/manuallyMarkedUnread flags — already the <i>correct</i> semantic for per-user group-chat preferences, no redesign needed. Message.senderUserId/senderUsername already exist on every message (types/index.ts:102-103) — group sender-name display (12.2) can be built from data the API already returns.
Structural blocker	Message.ciphertext/nonce are singular, non-nullable strings, and sendMessage carries exactly one recipientDeviceId (messageApi.ts:19-39) — the wire format structurally assumes one recipient per message row. This is the frontend half of Section 13's Phase 2 design, not a Phase 1 concern (Phase 1 ships no group messaging).
Sole conversation-creation entry point	UserSearchModal.tsx 's "Start Chat" button is confirmed (full-codebase check) as the <i>only</i> call site anywhere in the frontend that creates a conversation (conversationApi.createDirectConversation). This is precisely the component that must be re-gated behind the connection flow (12.2).
Latent caching bug this feature would trip over	cache/conversationListCache.ts 's sanitizeConversation() whitelist does not include type — a group conversation reloaded from the localStorage cache after a page refresh would silently lose its GROUP tag until the next full server refetch. Must be fixed in the same change that adds group support, not left as a follow-up.

AREA	VERIFIED FACT
Notification sender-name resolution	Background push/foreground notifications currently derive the displayed sender name via <code>getOtherParticipant</code> (App.tsx:1164-1167), not from the message's own <code>senderUsername</code> . For groups this must switch to the already-present <code>payload.senderUsername</code> field — a small, correctness-only fix that also simplifies the 1:1 path.
Already group-ready, no changes needed	Reaction aggregation (MessageBubble.tsx:370-385) already groups by emoji across an arbitrary number of users. Typing/read-receipt/presence WS fan-out is already member-list-based (Section 2.5). <code>appBadge.ts</code> is conversation-count-based, already type-agnostic. The Service Worker's push handler (public/sw.js:179-234) is entirely payload-driven with no conversation-shape assumption — no SW changes required for group push notifications.

12.2 Components to change (targeted, additive — not a rewrite)

COMPONENT	CHANGE
App.tsx	Introduce one new helper, <code>getConversationDisplayInfo(conv, currentUserId) → {mode: 'direct' 'group', title, avatarUrl, otherUser?}</code> , and migrate the 15+ <code>getOtherParticipant</code> call sites to it incrementally (not a single big-bang rewrite — see Section 19). Change <code>typingByConversation</code> from <code>Record<number,string></code> to <code>Record<number,string[]></code> so a group can show "Alice and Bob are typing..."; a 1:1 chat simply carries a one-element array. Add group WS event branches (12.4). Fix notification sender-name resolution (12.1).
ChatListSidebar.tsx	<code>getRecipient</code> replaced by <code>getConversationDisplayInfo</code> ; group conversations render their own name/avatar instead of being filtered out. Per-member pin/archive/unread logic (conversationList.ts:159-178) is reused unchanged.
MessageBubble.tsx	Add sender-name (and small avatar) rendering for non-self messages, shown only when <code>conversation.type === 'GROUP'</code> and gated by the already-computed <code>isGroupedWithPrev</code> flag (messageGroups.ts:72-82) so consecutive messages from the same sender don't repeat the label. Zero visual change to 1:1 bubbles. The reply-quote card's existing <code>replyToSenderUsername</code> rendering (MessageBubble.tsx:452-454) is the pattern to copy.
MessageInput.tsx	Phase 1: no change (group messaging UI stays hidden until Phase 2). Phase 2: new group-send branch resolving all current members' public keys (N calls to the existing <code>deviceApi.getUserPublicKeys</code> , already cached per-user for 10 minutes via <code>conversationCache.publicKeyMap</code>) and encrypting once per recipient (Section 13).
UserSearchModal.tsx	"Start Chat" becomes conditional on connection status: <code>ACCEPTED</code> → unchanged behavior; otherwise show "Send Connection Request" / "Request Pending" in its place. This is the direct UI expression of Section 3's governing principle.
cache/conversationListCache.ts	<code>sanitizeConversation</code> whitelist gains type (required fix, 12.1) plus group name/avatarUrl once those exist.
ForwardMessageModal.tsx	Migrate its local "other participant" logic to <code>getConversationDisplayInfo</code> so forwarding into a group shows the group's name in the picker, not a blank/incorrect row.

12.3 New components to introduce

COMPONENT	PURPOSE
CreateGroupModal.tsx	Group creation entry point. Member picker is scoped to the current user's <i>connections</i> (Section 10.1's <code>GET /connections</code>), not the global search used by <code>UserSearchModal</code> — structurally prevents Option B/C invitation risks (Section 7) from ever being reachable through this UI.

COMPONENT	PURPOSE
GroupInfoDrawer.tsx	Modeled directly on the existing ContactInfoDrawer.tsx : group name/avatar/description, member list with role badges, and role-gated action buttons (invite/remove/promote/demote/transfer/leave/delete) per the Section 8.2 matrix.
GroupInvitationsPanel.tsx	Incoming/outgoing group invitations, parallel in shape to a new ConnectionRequestsPanel.tsx needed for Section 4's connection requests — both are small, list-based, accept/reject/cancel UIs following the existing modal patterns already in the codebase (e.g. MuteChatModal.tsx , ClearChatConfirmDialog.tsx for structural reference).
ConnectionRequestsPanel.tsx	Same shape as above, for 1:1 connection requests (Section 4). Likely surfaced from NavigationRail.tsx/Sidebar.tsx alongside existing nav entries.

12.4 WebSocket event wiring

The existing WS handler in **App.tsx:958-1511** is a single large `useEffect` with a plain `if (event.type === '...') {...}` `else if (...)` chain (not an exhaustive `switch`) — a genuinely new, unhandled event type today would silently no-op rather than fail to compile. The new group event types (Section 11.2) should be added as additional branches in the same chain, following the same "patch the matching array/map entry, then update caches" pattern already used for every existing event. `WsEvent['type']` in **types/index.ts:163-179** must be extended with the new literal types first, or TypeScript won't flag a missed handler. Adding a `default/assertNever` guard at the same time is a low-cost safety net worth including, though not required.

13. Group E2EE Architecture

RECOMMENDATION — NOT IMPLEMENTED

THE CRYPTO FOUNDATION THIS DESIGN MUST BUILD ON (VERIFIED, BOTH SIDES OF THE WIRE)

ConnectX's E2EE is a **single, static, per-account ECDH P-256 identity keypair**, not a per-conversation or per-session key. `deviceSession.ts`'s "Account Master Identity Sync" reuses the exact same keypair across every device and every conversation a user has (`deviceSession.ts:15-16`), and the private key is fetched from / backed up to the server in plaintext PKCS#8 form via GET/POST `/users/me/identity-key` (`UserService.java:196-215`, `deviceSession.ts:23,76-85`). Every send today does exactly one ECDH derivation (my static private key + one peer's static public key) → AES-256-GCM. There is no ephemeral key exchange, no ratchet, and no forward secrecy in the current system, for 1:1 or otherwise. This is a pre-existing condition (flagged in the prior architecture audit as C-04/C-05/C-06), not something this document changes — but it directly bounds what "Group E2EE" can honestly claim, and, as a silver lining, it also means group E2EE does not need to solve Signal-style ephemeral session bootstrapping: the building block is simply "encrypt once per recipient's already-known static public key," which is exactly what 1:1 already does with $N=1$.

13.1 Three phases

PHASE	SCOPE	CRYPTO MODEL
PHASE 1	Group CRUD & membership (Sections 6–8)	None — no group messages can be sent yet. This is the primary scope of the roadmap in Section 18.
PHASE 2	Group messaging, interim/pragmatic crypto	Per-recipient fan-out: the sender encrypts the plaintext separately to each current member's static public key ($N-1$ ciphertexts per message), the same primitive used today, called N times instead of once.
PHASE 3	Production-grade group E2EE	Sender-key / ratchet upgrade (future) — explicitly gated on first closing the identity-key escrow gap (see 13.4).

13.2 Phase 2 in detail — per-recipient fan-out

DESIGN

Introduce a new child table, `message_recipient_payloads` (`message_id`, `recipient_user_id`, `ciphertext`, `nonce`) — see the dashed entity in Fig. 9.1. The parent messages row stays canonical for a group message (one `id`, one `sentAt`, one set of reactions/pins/stars/edits, exactly as today), while `ciphertext` becomes per-recipient in the child table instead of on the parent row. DIRECT messages are completely unaffected — they keep using `messages.ciphertext/nonce` exactly as today; this table is populated only for GROUP conversations. The backend's role is unchanged in kind: it stores and blindly relays N opaque ciphertexts instead of one, and never decrypts any of them — the same non-negotiable property as today (Section 2.4).

PROPERTY	PHASE 2 ANSWER
Group key	None — there is no shared symmetric group key at this phase, only N pairwise ciphertexts.
Member joining	Gains a public key to encrypt <i>future</i> messages to; nothing changes retroactively.

PROPERTY	PHASE 2 ANSWER
Member leaving/removal	Simply drops out of the recipient list for all future sends — the encryption-recipient-selection step (client reads current membership at send time) already, by construction, never encrypts a future message to a removed member. No explicit "rotation" action is needed for this guarantee at Phase 2, because there was never a shared secret to rotate.
New device / same account	Same as today's 1:1 behavior — the new device fetches the shared static identity key via the existing server-escrow mechanism. Inherits the existing gap (13.4); not made worse or better by groups.
Forward secrecy	None — matches the existing 1:1 posture exactly (no new regression, no new guarantee).
Past-message access for new members	Denied by default , as a natural consequence of the fan-out model: a new member's public key was never used to encrypt historical messages, so there is nothing for them to decrypt. An explicit "share history" feature would be new, opt-in functionality, not built here.
Future-message access for removed members	Denied at the encryption-recipient-selection step (client-side, at send time) — see "Member leaving/removal" above. This closes the "removed member accesses future messages" threat (Section 14) already at Phase 2, without needing key rotation, precisely because Phase 2 never had a rotatable shared secret in the first place.
Cost	$O(N)$ client-side encryptions and $O(N)$ storage per message. Acceptable for ConnectX's group sizes (a messaging app's typical small/medium groups, not a broadcast platform) and consistent with the non-goals implied by the existing architecture.

13.3 Phase 3 — future production-grade design (not built now)

A Megolm/Sender-Keys-style scheme: each member generates a symmetric "sender key" per epoch, distributed to other members using the Phase 2 pairwise primitive purely as the *key-distribution channel* (encrypt the small sender key to each recipient's static public key once per epoch); the sender key itself then symmetrically encrypts the bulk of message content, so steady-state cost drops to $O(1)$ per message with $O(N)$ cost only on membership change (add/remove) or explicit rotation.

PROPERTY	PHASE 3 ANSWER
Member removal	Triggers a key rotation: a new epoch's sender keys are generated and redistributed to the remaining members only — genuine forward secrecy for future messages relative to removal, even against a removed member who cached the old symmetric key (a real gap Phase 2 doesn't have to close because it never had a shared secret to cache).
New device / device key changes	Requires fetching the current epoch's sender key material for that device. Hard dependency : this only means something once device keys stop being a single server-escrowed static identity key — see Section 13.4.

13.4 The hard prerequisite

PHASE 3 CANNOT BE STRONGER THAN THE IDENTITY KEYS IT IS BUILT ON

Group E2EE's trust root is the same per-account identity key the 1:1 system already uses. A Phase 3 sender-key scheme layered on top of a server-escrowed, cross-device-shared static private key would improve group-specific properties (member-removal forward secrecy) while leaving the actual confidentiality root exactly as strong (or weak) as it is today — the server can already derive any pairwise shared secret it wants, sender keys or not, because it holds the private key. Recommendation: treat fixing the underlying identity-key escrow (C-04/C-05/C-06 from the prior architecture audit — non-extractable key generation, no server-side private-key storage, genuine per-device keys) as a prerequisite of Phase 3, not a parallel, independent workstream. Phase 1 and Phase 2 do not require this fix and are not blocked by it.

14. Security Threat Model

THREAT	BACKEND PROTECTION
Adding arbitrary users to a group	Invitation requires the inviter to hold OWNER/ADMIN role <i>and</i> an ACCEPTED connection with the invitee, both re-checked server-side at send time (Section 7).
Messaging a non-connection	<code>createOrGetDirectConversation</code> requires an ACCEPTED connection once Stage 2 enforcement is live (Section 5, 18).
Forged group invitations	Invitee identity is taken from the authenticated JWT principal on accept/reject, never from the request body; inviter's role is re-validated at accept time (not just trusted from send time), closing a TOCTOU window if the inviter was demoted/removed between sending and the invitee accepting.
Forged WebSocket group events	All group WS events (<code>GROUP_MEMBER_ADDED</code> , etc.) are server-generated exclusively from committed DB state via the existing <code>afterCommitExecutor</code> pattern; there is no client-writable STOMP destination that produces these events directly.
Unauthorized member removal / admin promotion	Role-hierarchy checks (Section 8.2) are enforced server-side on every call, never inferred from client-supplied role claims.
Group ID enumeration	Recommendation: <code>GET /groups/{id}</code> for a non-member returns 404 <code>GROUP_NOT_FOUND</code> , not 403 — a 403 would confirm the group ID exists even to an outsider; 404 does not.
Unauthorized message retrieval	Existing <code>existsByConversationIdAndUserIdAndDeletedAtIsNull</code> membership check (Section 2.3) already generalizes correctly to N-member groups with no code change.
Block bypass	Block check enforced server-side at every DIRECT send, independent of any client-side UI state (Section 4.3).
Replay of invitations	Invitations are authenticated-endpoint actions (accept requires being logged in as the invitee), not link/token-based — there is no bearer token to intercept or replay, unlike the existing OTP flows. A partial unique index prevents duplicate PENDING invitations.
Stale membership on the client	REST reads always reflect live DB state; the removal WS event forces an immediate client-side correction for connected sessions (Section 11.2).
Removed member continuing to receive messages	Layered: (a) SUBSCRIBE-time auth fix (Section 11.1) blocks new/reconnected subscriptions immediately; (b) an explicit removal event tells the still-connected client to locally unsubscribe; (c) Phase 3 key rotation (13.3) additionally denies decryption even if a transport-level message slipped through during the bounded exposure window.
New member accessing past encrypted messages	Denied by construction at Phase 2 — a new member's key was never used to encrypt historical ciphertext (Section 13.2).
Removed member accessing future messages	Denied by construction at Phase 2 — the encryption-recipient list is read from current membership at send time (Section 13.2); Phase 3 rotation adds protection against a cached shared secret once one exists.
Ownership deadlock / dual-OWNER race	Leave is blocked for an OWNER until ownership is transferred (or the group is deleted if they're the sole member); promote/demote/transfer run inside a single transaction with the current role re-read and re-checked immediately before the write, following the existing <code>REQUIRES_NEW</code> -isolated-write pattern already proven for the reaction/star races (Section 15).
Concurrent mutual connection requests (A→B and B→A simultaneously)	Must resolve to exactly one connections row, not two and not a raw constraint-violation 500 — see Section 17 for the required regression test, using the same proven concurrency-handling pattern as the existing reaction/star race fixes.

15. Performance Considerations

PRECEDENT ALREADY SET IN THIS CODEBASE

ConnectX has already been through two rounds of N+1 elimination (the conversation-list `findLatestMessagesForConversations` batching, the reaction/media batch-fetch in `getConversationMessagesPaged`) — the pattern to follow for every new list-shaped group endpoint is well established: one JOIN FETCH query per screen, never a query-per-row loop.

CONCERN	MITIGATION
GET /groups/{id}/members	Single JOIN FETCH query, mirroring the existing <code>findByConversationIdWithUsers</code> — never one query per member.
Connections/blocks hot-path checks	The connections table's unique index on (user_id_low, user_id_high) (Section 9.1) makes "are these two connected" an O(1) indexed lookup, matching the shape of every other hot-path authorization check already in the codebase — deliberately not a status-filtered scan of connection_requests.
Block checks do not apply per-pair inside groups	Blocking is enforced only for DIRECT sends (Section 4.3) — a group send never checks all $C(N,2)$ member pairs for mutual blocks; that would be both expensive and outside the actual threat model (group membership itself, not pairwise connections, is the permission for group messaging).
Conversation-list enrichment	<code>findLatestMessagesForConversations</code> is already conversation-type-agnostic and requires no changes to remain N+1-free once groups are mixed into the same list.
Group member growth	All existing per-conversation indexes (idx_conv_user, idx_msg_conversation) are already composite on conversation_id and generalize to larger member counts with no index changes.
Phase 2 fan-out storage/compute	O(N) by design (Section 13.2) — acceptable for typical group sizes; flagged here as the one place group support meaningfully changes the app's per-message cost profile, worth monitoring in practice once real group sizes are known.

16. Migration Strategy

RECOMMENDATION — OPERATIONAL ROLLOUT PLAN, BUILDING ON SECTION 5'S DATA-PRESERVATION DESIGN

16.1 Deployment ordering

1. **Schema-only deploy** (Stage 0): new tables (`connection_requests`, `connections`, `user_blocks`, `groups`, `group_invitations`) and two new nullable columns on `conversation_members`. Purely additive — safe under a rolling restart, no existing query is affected because nothing yet reads or writes the new tables/columns.
2. **Run the backfill** (Section 5) and verify: the count of new connections rows must equal the count of distinct user pairs sharing a DIRECT conversation, computed independently as a sanity check before proceeding.
3. **Deploy connection/blocking API code** (Stage 1, 3) with the "connection required to start a DM" enforcement **behind a config flag** (`connectx.connections.enforce=false` initially) — lets the new endpoints go live and be exercised without yet changing existing behavior.
4. **Flip enforcement on** (Stage 2) once the backfill is confirmed complete and the new endpoints have been exercised in the target environment. Monitor the rate of 403 `NOT_CONNECTED` rejections immediately after the flip — a spike indicates a backfill gap, not legitimate new-stranger blocking (existing pairs should never hit this error).
5. **Groups (Stage 4–9)** layer on top once 1–4 are stable, since group invitations reuse the same connections table.

16.2 Rollback

Because every schema change through Stage 6 is additive (new tables, nullable columns), rolling back to a previous application JAR is always safe — the new tables simply go unread/unwritten again, with no destructive schema rollback ever required. If enforcement (Stage 2) causes unexpected breakage, the fastest rollback is flipping `connectx.connections.enforce` back to `false` (a config change, not a redeploy); a full code rollback remains available as a second line of defense.

17. Testing Strategy

FOLLOW THE EXISTING, PROVEN PATTERN

Every isolation/race-sensitive test in this codebase runs as `@SpringBootTest @ActiveProfiles("test")` against a real MySQL instance, not H2 (H2's differing default isolation level was explicitly why the reaction-visibility bug went undetected by unit tests — see prior investigation notes). The Groups work should follow the same rule for anything connection/role/membership-race related.

17.1 New tests required

TEST	WHY
Backfill idempotency & correctness	Running the backfill twice must not duplicate rows; the resulting connections count must exactly match independently-computed distinct DIRECT pairs.
Concurrent mutual connection request	Alice→Bob and Bob→Alice requests fired simultaneously must resolve to exactly one connections row — recommend the same REQUIRES_NEW-isolated-insert-with-catch pattern already proven and tested in ReactionRaceIntegrationTest/StarRaceIntegrationTest .
Concurrent promote/demote/transfer-ownership	Must never leave a group with zero or two OWNERS.
Block enforcement	Send-after-block rejection in both directions; existing conversation history remains visible; connection/invitation requests between a blocked pair are rejected.
Invitation-requires-connection	Inviting a non-connection is rejected even if the inviter is a valid OWNER/ADMIN.
Full role-permission matrix (Section 8.2)	Parameterized test asserting every cell of the matrix, including the "ADMIN cannot remove ADMIN" and "cannot remove OWNER" edge cases.
WS SUBSCRIBE-time authorization	A non-member's attempt to subscribe to <code>/topic/conversation/{id}</code> must be rejected. This closes a gap the prior architecture audit found but never got a regression test for — valuable independent of Groups, and a hard prerequisite of Section 11.1.
Standing regression gate	The existing suite (ReactionVisibilityTest , StarRaceIntegrationTest , ConversationDeletionCleanupTest , OtpRateLimitIntegrationTest , etc.) must pass unchanged at the end of <i>every</i> stage in Section 18, not just once at the end of the whole initiative — this is how "existing messaging must not break" gets verified continuously rather than assumed.

17.2 Frontend

No browser-automation tooling exists in this repository today (no Playwright/Puppeteer in devDependencies, confirmed). Until that changes, the practical verification method is the same hand-rolled CDP/headless-Chrome approach already used successfully for the stuck-connecting WebSocket investigation. At minimum, add focused unit tests around the new `getConversationDisplayInfo` helper (Section 12.2) given how many call sites will come to depend on it.

18. Phased Implementation Roadmap

Eleven stages, each independently deployable and independently rollback-able. Existing messaging behavior is verified unbroken at the end of every stage (Section 17.1).

Stage 0 — Architecture & database preparation

Files affected	New Flyway migration scripts (recommended, Section 5.3); no application code changes.
Database changes	Additive: <code>connection_requests</code> , <code>connections</code> , <code>user_blocks</code> , <code>groups</code> , <code>group_invitations</code> ; <code>conversation_members</code> gains nullable <code>role</code> , <code>invited_by_user_id</code> .
API / Frontend / WebSocket changes	None — nothing reads or writes the new schema yet.
Security	No new attack surface (nothing is reachable yet).
Tests	Schema-creation smoke test; confirm existing suite still passes unmodified.
Rollback	Trivial — unused tables/columns, safe to leave in place or drop.

Stage 1 — Connection system

Files affected	New connection backend package (entity/repo/service/controller/dto); new <code>connectionApi.ts</code> ; new <code>ConnectionRequestsPanel.tsx</code> .
Database	None beyond Stage 0.
API	Section 10.1 endpoints go live.
Frontend	<code>UserSearchModal.tsx</code> gains the "Send Connection Request" action (Section 12.2), but "Start Chat" still works unconditionally — enforcement is not flipped on yet (16.1).
Security	Request/accept/reject/cancel authorization (Section 10.1's authz column).
Migration	None yet.
Tests	Full request lifecycle; concurrent-mutual-request race (17.1).
Rollback	Disable the new endpoints/UI; schema stays, unused.

Stage 2 — Existing-chat migration + authorization turn-on

Files affected	Backfill script/runner; <code>ConversationService.createOrGetDirectConversation</code> gains the connection check, behind the config flag (16.1).
Database	Backfill writes <code>connections/connection_requests</code> rows only (Section 5) — zero writes to <code>messages/conversations/ciphertext</code> .
API	<code>POST /conversations/direct</code> starts returning 403 <code>NOT_CONNECTED</code> for non-connections once the flag flips.
Frontend	<code>UserSearchModal.tsx</code> 's "Start Chat" becomes conditional (12.2).
Security	Closes the core gap from Section 3.
Migration	The critical stage — backfill correctness verified per 16.1 <i>before</i> the flag flips.
Tests	Backfill idempotency/correctness (17.1); regression on every existing DIRECT-conversation flow.
Rollback	Flip the config flag back to false — no redeploy needed.

Stage 3 — Blocking

Files affected	New user_blocks repo/service/controller; block checks added to <code>MessageService.sendMessage</code> and <code>WebSocketMessageController</code> ; new block/unblock UI.
Database	None beyond Stage 0.
API	Section 10.2 endpoints.
Frontend	Block/unblock actions surfaced from <code>ContactInfoDrawer.tsx</code> .
Security	Section 4.3's full effects table enforced.
Tests	Block-enforcement suite (17.1).
Rollback	Disable endpoints/UI; no schema rollback needed.

Stage 4 — Group CRUD (Phase 1 E2EE scope)

Files affected	New group backend package; groups table live; <code>ConversationType.GROUP</code> activated; <code>CreateGroupModal.tsx</code> , <code>GroupInfoDrawer.tsx</code> ; <code>getConversationDisplayInfo</code> introduced (12.2).
Database	groups table populated for the first time.
API	Section 10.3 group CRUD/leave/transfer endpoints.
Frontend	<code>ChatListSidebar.tsx</code> stops filtering out groups (12.2); no group <i>messaging</i> UI yet.
WebSocket	None yet — no group messages exist.
Security	Role checks (Section 8) enforced from the start.
Tests	Group lifecycle + role-permission matrix (17.1).
Rollback	Additive; disable group-creation UI if needed.

Stage 5 — Group invitations

Files affected	group_invitations live; <code>GroupInvitationsPanel.tsx</code> .
Database	group_invitations populated.
API	Section 10.4.
Security	Connection-required-to-invite (Section 7) enforced server-side.
Tests	Invitation-requires-connection rejection test (17.1); replay/duplicate-invitation tests.
Rollback	Additive.

Stage 6 — Group permissions hardening

Files affected	Promote/demote/remove/transfer endpoints and their concurrency guards.
Security	Ownership-deadlock and dual-OWNER race protections (Section 14).
Tests	Concurrent promote/demote/transfer races (17.1).
Rollback	Additive.

Stage 7 — Group messaging (Phase 2 E2EE)

Files affected	Hard prerequisite, must land first in this stage: the WS SUBSCRIBE-time authorization fix (Section 11.1) — not deferrable. Then: message_recipient_payloads table + write path; MessageInput.tsx group-send branch; MessageBubble.tsx sender-name rendering.
Database	New message_recipient_payloads table (Section 13.2); zero changes to the existing messages table or DIRECT ciphertext path.
API / WebSocket	Reuses existing message endpoints/destinations unchanged (Section 10.5); new group WS events (Section 11.2).
Security	Section 11.1's fix is the gating condition for this entire stage.
Tests	WS SUBSCRIBE-authorization regression test (17.1); fan-out encryption correctness (each current member's payload decrypts, a removed member's does not receive future payloads).
Rollback	Group-send UI can be disabled independently of the WS fix, which should remain permanent regardless.

Stage 8 — Group E2EE hardening (Phase 3 planning)

Scope	Design spike for the sender-key/ratchet upgrade (Section 13.3), explicitly gated on first addressing the identity-key escrow prerequisite (13.4) — a separate initiative in its own right, sequenced here rather than blocking Stages 4–7.
Rollback	N/A — planning stage.

Stage 9 — Notifications / PWA integration

Files affected	Backend push payload gains group name/sender fields; App.tsx notification sender-name fix (12.1, switch from getOtherParticipant to payload.senderUsername).
Frontend / SW	public/sw.js requires no changes — confirmed already payload-driven and conversation-id-generic (12.1).
Tests	Push payload shape for group messages; notification click routing to the correct group conversation.
Rollback	Additive payload fields only.

Stage 10 — Testing / security audit

Scope	Full regression pass across every stage's tests (17.1); a dedicated security review of the new connection/block/group/invitation/WS surface, in the same style as the prior 2026-08-16 architecture audit.
Rollback	N/A — verification stage.

19. Risks and Alternatives

DECISION	ALTERNATIVE CONSIDERED	WHY REJECTED / DEFERRED
Two-table connections model (4.1)	Single table carrying both workflow and final state	Simpler schema, but mixes audit history with the hot-path "are we connected" lookup that will run on every DM/invite; rejected for query-shape consistency with the rest of the codebase. Reasonable if the team strongly prefers fewer tables.
Group invitations require a connection (Option A, Section 7)	Option B (open) / Option C (configurable privacy)	B reintroduces the exact gap this document closes for DMs. C is a reasonable future extension once real usage patterns exist, deliberately deferred to avoid speculative complexity now.
All group members join via invitation-acceptance, even founding members (6.4)	WhatsApp-style: creator's initial picks are added directly, no acceptance step	Friendlier UX, but reintroduces a narrow version of "added without consent" for the founding member set and adds a second membership code path. Rejected for Phase 1; could revisit later scoped strictly to existing connections.
Flyway adoption (5.3)	Guarded idempotent ApplicationRunner bean, no new dependency	Lower short-term friction, but leaves the "no migration history" latent operational risk unresolved (a risk that exists independent of Groups). Noted as the fallback if introducing Flyway is judged out of scope.
Fix WS SUBSCRIBE-time auth (11.1)	Remove the public /topic/conversation/{id} broadcast entirely, rely only on per-user queues	Considered and rejected once the frontend trace confirmed the topic is genuinely load-bearing for live secondary signals (typing/reactions/edits/pins) while a conversation is open on screen (Section 12.1) — removing it would require redesigning that live-update path, a larger change than fixing the authorization gap directly.
Phase 3 group E2EE sequencing (13.4)	Build sender-keys/ratchet immediately alongside Phase 2	Would layer real engineering effort on top of a trust root (server-escrowed identity key) that undermines the payoff; risk of scope creep and a false sense of security. Sequenced after the identity-key fix instead.
getOtherParticipant centralization (12.2)	Single big-bang rewrite of all four call sites at once	Higher regression risk to the currently-working 1:1 path for a UI-layer refactor. Recommended instead as an additive helper migrated in incrementally, consistent with the instruction not to unnecessarily refactor working code.

20. Final Recommended Architecture



Fig. 20.1 — the four states this entire document keeps distinct, per Section 3's governing principle. Every arrow requires an explicit, server-enforced acceptance step; none is implied by the state before it.

20.1 What exists today

A working, reasonably mature DIRECT-only messaging app: JWT auth, STOMP/WebSocket real-time delivery, pairwise ECDH+AES-GCM encryption for TEXT messages, read/delivery receipts, typing indicators, reactions, pins, edits, media, Web Push, and a hand-rolled PWA layer — all built on a Conversation/ConversationMember model that already, if unintentionally, anticipated groups (ConversationType.GROUP exists on both backend and frontend, unused). Search-to-message has no permission gate. No connection, block, or group concept exists in the schema.

20.2 What must change

- New relationship layer: connection_requests, connections, user_blocks.
- Enforcement of that layer at every DM-creation, messaging, and (once built) group-invitation boundary — server-side, never trusted from the client.
- Activation of the dormant GROUP conversation type, plus groups and group_invitations tables and a role column on conversation_members.
- The WebSocket SUBSCRIBE-time authorization fix (Section 11.1) — mandatory before group messaging, valuable independent of groups.
- A handful of targeted frontend changes centered on one new helper (getConversationDisplayInfo) replacing four duplicated "assume exactly one other member" implementations.

20.3 What must not change

Existing conversation IDs, existing message rows and their ciphertext, the DIRECT send/receive/receipt/typing/presence pipeline, the existing E2EE algorithm and key model (Phase 1 & 2 touch neither), Web Push delivery, and authentication. Every migration in this document is additive; every enforcement change is staged behind a reversible flag.

20.4 Recommended architecture, in one paragraph

Extend the existing Conversation/ConversationMember model rather than building a parallel one; gate all new relationships (connection, block, group membership) behind explicit server-enforced acceptance, never inferred from discovery; migrate existing DIRECT conversations into ACCEPTED connections losslessly and invisibly to current users; fix the pre-existing WebSocket authorization gap as a prerequisite of group messaging rather than a parallel task; and phase group E2EE so that Phase 1 (membership) and Phase 2 (pairwise fan-out messaging) ship on the existing crypto primitive with no new trust assumptions, while Phase 3's stronger guarantees are explicitly sequenced behind fixing the identity-key escrow issue that the current 1:1 system already has.

20.5 Recommended implementation order

Stages 0–3 (relationships, migration, blocking) should ship and stabilize in production before Stage 4 (group CRUD) begins — every later stage depends on the connections table being correct and enforced. Stage 7 (group messaging) cannot

begin before its WS-authorization prerequisite lands. Stage 8 (Phase 3 crypto) is intentionally decoupled from the rest and can proceed on its own timeline.

20.6 Biggest security risk

Shipping group invitations (Stage 5) before the WS SUBSCRIBE-time authorization fix (Stage 7's prerequisite) would multiply today's single worst unresolved finding: a hole that leaks one conversation's traffic today would leak an entire group's traffic, to anyone who can guess a conversation ID, including former members. This is the one ordering constraint in this document that must not be relaxed.

20.7 Biggest migration risk

An incomplete or incorrect backfill (Section 5) run before enforcement is flipped on (Stage 2) would lock existing users out of conversations they already have — the count-verification step in Section 16.1 and the config-flag rollback are the two safeguards against this being anything worse than a delayed rollout.

End of document. Prepared from direct inspection of the ConnectX repository (backend: full trace of entities/services/controllers/WebSocket/security; frontend: full trace via dedicated exploration of App.tsx, WebSocketClient.ts, UserSearchModal.tsx, crypto/*, cache/*, and PWA notification handling) on 2026-08-17. No source code, database schema, or migrations were modified in producing this document.